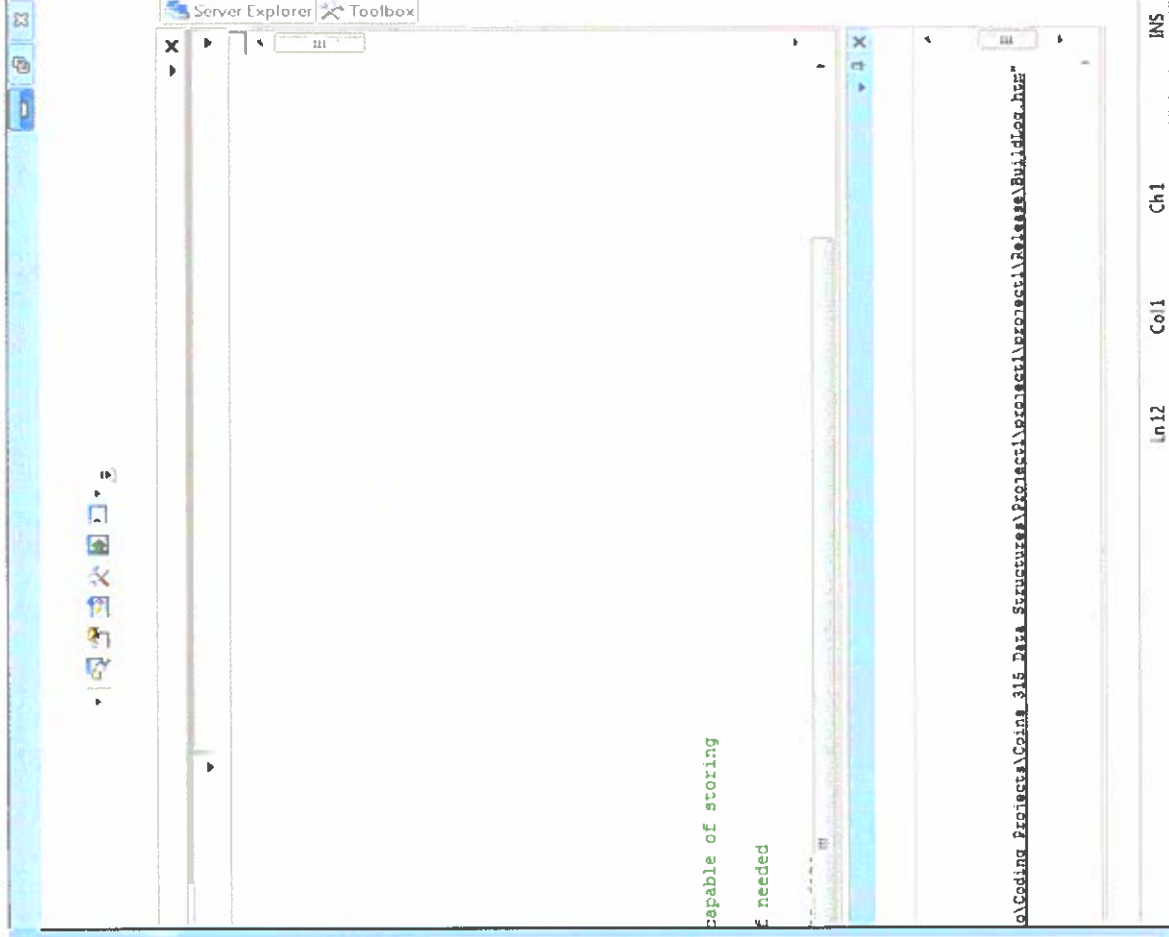
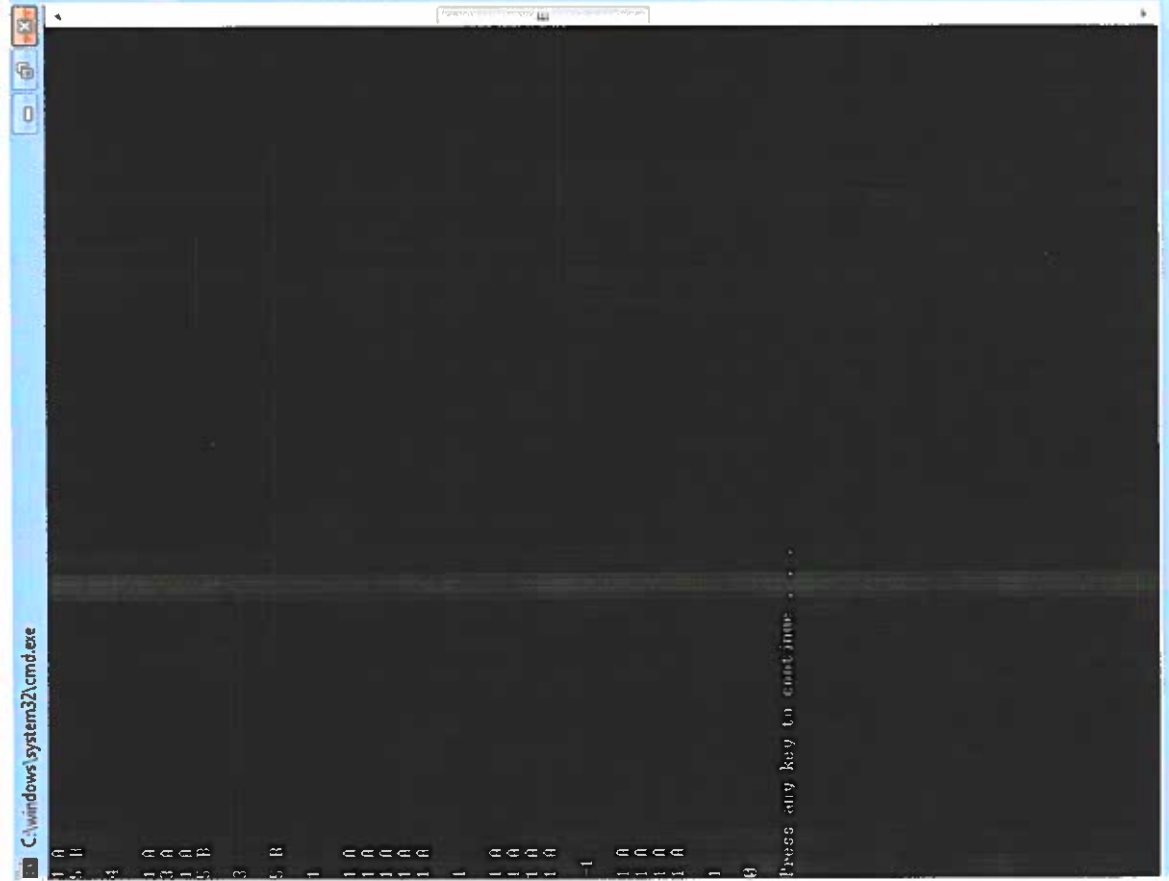


Programming Projects

1. Project1 – Indexed Multi-list. Created for Data Structures. Written in C++ and edited in the Microsoft Visual Studio 2005 IDE. Grade: 92.
2. Daytona – JavaApplet. Was created as the final project for Java I. Was written in java and created via the Netbeans IDE. Grade: 100. The code uses double buffering to move the cars across the screen at a speeds based of f of the given input and the fps of the screen.
3. termProject – SQL Database Website. Created for Applied Networking as the term project. It was written via SQL in Microsoft Visual Studio 2005 IDE. Grade: 85. The program uses databases to allow users to search and purchase products. The website also manages users and logins.
4. Project 2 – Indexed Multi-list. Created as the second Data Structures project. Written in C++ and edited in the Microsoft Visual Studio 2005 IDE. Grade: 85.



```

// Eric Foster
// IndexedMultiList
// January 20th, 2009
// January 20th, 2009
// Data Structures
// Grade 92
// A simple indexed multilist.
// testIndexedMultiList.cpp

#include <iostream>
#include "IndexedMultiList.h"

using namespace std;

void main()
{
    IndexedMultiList current;//instantiate new object.

    cout<<current.size()<<endl<<endl;
    //Should output 0, since array is empty;

    cout<<current.empty()<<endl<<endl;//Attempt to print 1, as array is
empty.

    for(int i=0; i<5;i++)
    {
        current.insert(i,'A',1);//fill all spots to
        //1A
    }

    cout<<current.empty()<<endl<<endl;
    //Should output 0 because array isn't empty
    current.print();
    //Should print all 1A because they were inserted.
    cout<<endl;

    cout<<current.change(0,2)<<endl<<endl;
    //Should output 1 because index was valid.
    current.print();
    cout<<endl;
    //Should change array to 3A, followed by all 1A.

    cout<<current.change(1,0)<<endl<<endl;
    //Should output 1 because index was valid.
    current.print();
    cout<<endl;
    //Array should remain unchanged.

    cout<<current.change(3,2)<<endl<<endl;
    current.print();
    cout<<endl;
    //First and 3rd should be 3A, rest all 1A.

    cout<<current.change(2,-1)<<endl<<endl;
    //Should return 1, had a valid index.
    current.print();
}

```

```

cout<<endl;
//First and 2nd should now be 3A, rest should
//all be 1A.
//Should also only go to index of 4 since item was
//removed from array.

cout<<current.remove('A')<<endl<<endl;
//Should output 1, and remove first A.
current.print();
cout<<endl;
//2nd spot should now be 3A, rest all 1A. Go to a
//index of
//3, since there is now another empty spot.

cout<<current.remove('B')<<endl<<endl;
//Should output -1, no B exists.
current.print();
cout<<endl;
//Nothing should have changed.

cout<<current.find('A')<<endl<<endl;
//Should print index of 0, A exists.
current.print();
cout<<endl;
//Nothing should have changed.

cout<<current.find('B')<<endl<<endl;
//Should print -1, B isn't in array.

cout<<current.countItem('A')<<endl<<endl;
//Should print out 3, A exists 3 times in array.

cout<<current.countItem('B')<<endl<<endl;
//Should output 0, b doesn't exist.

cout<<current.find('B')<<endl<<endl;
//Should output 1, item exists.

cout<<current.totalItemCount('B')<<endl<<endl;
//Should output 0, item doesn't exists.

cout<<current.totalItemCount('A')<<endl<<endl;
//Should output 7, total A's in array.

current.insert(0, 'B', 5);
current.print();
cout<<endl;
//First space should now have 5B.

cout<<current.totalItemCount('B')<<endl<<endl;
//Should output 5, there are 5 B's in array.

cout<<current.countItem('B')<<endl<<endl;
//Should now output 1, B exists in first spot.

cout<<current.size()<<endl<<endl;
//Array is now 10, so should output 10.

```

```

cout<<current.update('C',0)<<endl<<endl;
//doesn't exist, should output 0.

cout<<current.update('B',0)<<endl<<endl;
//Adding zero, shouldn't change anything.
//Should output 1 since it exists.

cout<<current.update('B',-1)<<endl<<endl;
//Should return 1, should change to 4B.
current.print();
cout<<endl;

cout<<current.update('B',-4)<<endl<<endl;
//Should return 1, should remove 4B from list;
//Now index up to 2;
current.print();
cout<<endl;

//Just adding something else for testing...not a test
//case.
current.insert(3,'B',5);
current.print();
cout<<endl;

cout<<current.removeAll('C')<<endl<<endl;
//Should output 0, no instances of c
//C doesn't exist shouldn't do anything.
current.print();
cout<<endl;

cout<<current.removeAll('A')<<endl<<endl;
//Should output 3, A exist in 3 different spots.
//Should remove all A's from entire array leaving
//only 5B at beginning.
current.print();
cout<<endl;

cout<<current.removeAll('B')<<endl<<endl;
//Should output 1, 5B is all that exists.
//Array is now empty;
for(int i=0; i<5;i++)
{
    current.insert(i,'A',1);//fill all spots to
    //1A
}
current.print();
cout<<endl;
//should out put 5 1A's

cout<<current.remove(0)<<endl<<endl;
//Should output 1, 1A exists at spot.
current.print();
cout<<endl;
//Should print array of 4 1A's.

cout<<current.remove(5)<<endl<<endl;

```

```

current.print();
cout<<endl;
//Invalid index, should output -1.
//Shouldn't change array.

//Couldn't Make the getAt to compile.

//cout<<current.getAt(1,'B')<<endl<<endl;
//Should output 1, since spot is in use;

//cout<<current.getAt(9,'B')<<endl<<endl;
//Should output 0, since spot isn't in use;

cout<<current.countPosition(0)<<endl<<endl;
//Should return 1, since 1A in spot.

cout<<current.countPosition(6)<<endl<<endl;
//Should return 0, invalid index.

```

```

}

```

```

// Eric Foster
// IndexedMultiList
// January 20th, 2009
// January 20th, 2009
// Data Structures
// Grade 92
// A simple indexed multilist.
// IndexedMultiList.cpp

```

```

#include <iostream>
#include "IndexedMultiList.h"

```

```

using namespace std;

```

```

int emptySpaces=20;//keeps track of empty spaces.

```

```

IndexedMultiList::IndexedMultiList()
{

```

```

} //constructor for IndexedMultiList that creates the iml
//array of structs.

```

```

//accessor to determine if the array is empty.

```

```

bool IndexedMultiList::empty()
{

```

```

        if(emptySpaces==20)
        //using emptySpaces to increment via the remove method.
        {
            return true;
            //true if 20 items, which is all
            //of the items, have been removed or don't exist.
        }
        else
        {
            return false;
            //returns false if less than 20 have been removed.
        }
    }

int IndexedMultiList::uniqueSize()
{
    if(emptySpaces==20)
    {
        return 0;
        //return 0 if all spaces are empty;
    }
    else
    {
        return (20-emptySpaces);
        //return the array size minus empty spaces;
    }
}

int IndexedMultiList::size()
{
    int count=0;
    for(int i=0; i<uniqueSize(); i++)
    {
        count+=(iml[i].howMany);
        //increment count to add the number of times that letter occurs.
    }
    return count;//return the running count.
}

int IndexedMultiList::find(char val)
{
    int toReturn=-1;
    for(int i=0; i<uniqueSize(); i++)
    {
        if(iml[i].letter==val)
        {
            toReturn=i;
        }
    }
    return toReturn;
}

int IndexedMultiList::countItem(char val)
{
    int count=0;//keeps running total of locations to be returned.
    for(int i=0; i<uniqueSize(); i++)
    {
        if((iml[i].letter)==val)//if current val is found.
        {

```

```

        count++; //add one location to count.
    }
    }
    return count; //return the running total.
}

int IndexedMultiList::totalItemCount(char val)
{
    int count=0; //keeps running total of items to be returned.
    for(int i=0; i<uniqueSize(); i++)
    {
        if((iml[i].letter)==val)
        //if current val is found.
        {
            count+=(iml[i].howMany);
            //add howMany are in the position to count.
        }
    }
    return count; //return the running total.
}

int IndexedMultiList::countPosition(int index)
{
    if(index>=0 && index<uniqueSize())
    //if 0<=index<=DEFAULT_MAX_ITEMS then index is valid.
    {
        return iml[index].howMany;
        // return the number of items at that position.
    }
    else
    {
        return 0;
        // invalid index, return 0.
    }
}

bool IndexedMultiList::getAt(int index, char &out)
{
    if((index<uniqueSize()))
    //if value exists at least once in that spot, it is in use.
    {
        out=(iml[index].letter); //out = that val.
        return true;
    }
    else
    {
        return false;
        //spot isn't in use, return false.
    }
}

void IndexedMultiList::print()
{
    for(int i=0; i<uniqueSize(); i++)
    {
        cout<< iml[i].howMany<<" " << iml[i].letter<<endl;
        //print the number, then letter in form such as 1D;
    }
}

```



```
)
```

```
bool IndexedMultiList::insert(int index, char val, int count)
{
    if((index>=0) && (index<=uniqueSize()) && (count>=1))
//validate index and assure count is at least one.
    {
        iml[index].letter=val;//change struct letter.
        iml[index].howMany=count;//change struct howMany.
        emptySpaces--;//Removed an empty space;
        return true;//return true showing it worked.
    }
    else
    {
        return false;//return false show it failed.
    }
}
```

```
bool IndexedMultiList::change(int index, int amount)
{
    if(index>=0 && index<uniqueSize())//if valid index
    {
        (iml[index].howMany)+=amount;//add amount to index
        if(iml[index].howMany<=0)
//If position isn't in use after addition.
        {
            remove(index);
        }
        return true;//return true because index was valid.
    }
    else
    {
        return false;//return false, index was invalid.
    }
}
```

```
bool IndexedMultiList::update(char val, int amount)
{
```

```
    int first=find(val);
    if(first>=0)
    {
        change(first, amount);
        return true;
    }
    else
    {
        return false;
    }
}
```

```
bool IndexedMultiList::remove(char val)
```

```
{
    if(find(val)==-1)
    {
        return false;
    }
}
```

```

        int index=(find(val));
        if(remove(index)!=-1)
        {
            return true;
        }
        else
        {
            return false;
        }
    }

int IndexedMultiList::remove(int index)
{
    int indexSaver=-1;//hold amounts in position.
    if((index>=0)&&(index<uniqueSize()))
    {
        indexSaver=(iml[index].howMany);//save amount in position.
        int i=index;
        emptySpaces++;
        for(int i=index; i<uniqueSize(); i++)
        {
            iml[i]=iml[i+1];
        }
    }
    return indexSaver;
}

int IndexedMultiList::removeAll(char val)
{
    int count=0;//keep track of vals removed.
    for(int i=0; i<uniqueSize();i++)
    {
        remove(val);
        count++;
        if(remove(val)==true)
        {
            count++;
            //if value was removed, iterate count to keep
            //total of locations removed.
        }
    }
    return count;
}

// Eric Foster
// IndexedMultiList
// January 20th, 2009
// January 20th, 2009
// Data Structures
// Grade 92
// A simple indexed multilist.
// IndexedMultiList.h

#ifndef H_IndexedMultiList
#define H_IndexedMultiList

```

```

using namespace std;
const int DEFAULT_MAX_ITEMS=20;

class IndexedMultiList {
public:
//Constructors/destructors
    IndexedMultiList();//creates an empty list capable of storing
//DEFAULT_MAX_ITEMS items
    //~IndexedMultiList(); //destructor ONLY if needed

    //struct multiList//creating a struct for the data.
    //{
    //    char letter;//the letter stored.
    //    int howMany;//amount of letter in that spot.
    //};

//accessors
    bool empty();//returns true iff (if and only if) list is empty
    int uniqueSize();//returns number of positions in the list in use
    int size(); //returns total number of items in list using counts
//If 2 A, 3 C, 1 D are inserted into an IndexedMultiList iml then
//impl.uniqueSize() would return 3 and impl.size() would return 6.
    int find(char val);
//returns index of first position with val in it or -1 if not found
    int countItem(char val);
//returns number of locations val is in the list (vacuously 0 if not //found)
    int totalItemCount(char val);
/*returns number of val in the list (vacuously 0 if not found)
Example: there are 3 A's in the first position, 6 B's in the second and 4 A's
in the third position.
find('A') returns 0, find('B') returns 1 and find('C') returns -1
countItem('A') returns 2 (it is in the first and third location)
countItem('B') returns 1, countItem('C') returns 0
totalItemCount('A') returns 7, totalItemCount('B') returns 6,
totalItemCount('C') returns 0
*/

    int countPosition(int index);//returns number of items at position
index (0 based) or 0 if index is invalid
    bool getAt(int index, char &out);
//if position index is in use, out is set to char at that position and true
is returned, otherwise false is returned.
    void print();
//prints out the content of the list in order by position, see below

//mutators
    bool insert(int index, char val, int count);
/*insert count number of val into the list at position index
returns true iff val was able to fit in list and index was valid and count
>=1
valid indices range from 0 up to unique size (inclusive). So for an empty
list, 0 is the only valid index.
insert should do nothing if index is invalid
If the list has 3 A's in the first position, then 0 and 1 are the only valid
indices.
*/

```

```

    bool change(int index, int amount);
/*add amount to item in position index
returns true iff index was valid
change should do nothing if index is invalid
amount could be negative, if adding amount to the current count results in a
count of <= 0 then position should be removed
For example, if there are 3 A's at index 0 and 2 B's at index 1 in the list
iml then
iml.change(0, -4)
would remove all the A's and the 2 B's would be at index 0.
*/

```

```

    bool update(char val, int amount); //finds val in list and adds amount
(which could be negative)
//return true iff found, works similarly to change but is based on the
//character to find instead of the position. If val is in multiple
//locations within the list, then only change the first location

```

```

    bool remove(char val); //removes 1 instance of val from list,
//returns true iff val was in list, if val is in multiple locations,
//then only remove from the first location

```

```

    int remove(int index); //remove position at index, returns count //of
items removed or -1 if index not in use

```

```

    int removeAll(char val); //removes all instances of val from list
//returns number of items removed, which could be 0, if val is in //multiple
locations it should remove only the first location

```

```

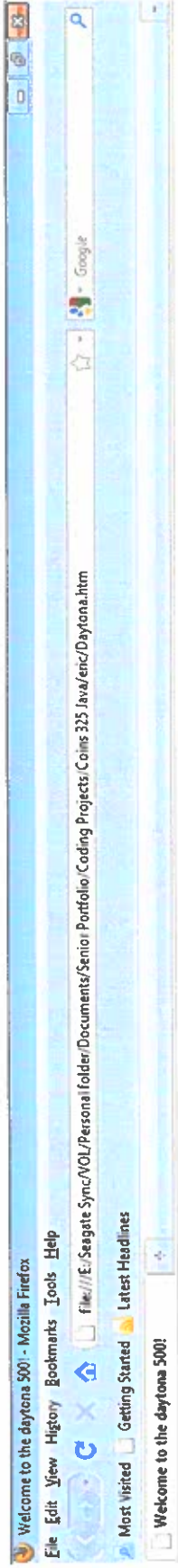
private:
//put whatever methods or data you need in here,
//you must put the 1-dimensional array here
    struct multiList
    {
        char letter;
        int howMany;
    };
    multiList iml[DEFAULT_MAX_ITEMS];
};

```

```

#endif

```



Start Finish Race Speed for the 24 car 1 Speed for the 8 car 2 Speed for the 99 car 3 Speed for the 20 car 4

Please enter the speeds from 1-40 and press start to begin the animation. Press finish race if you wish to end the race.



Welcome to the daytona 500! - Mozilla Firefox

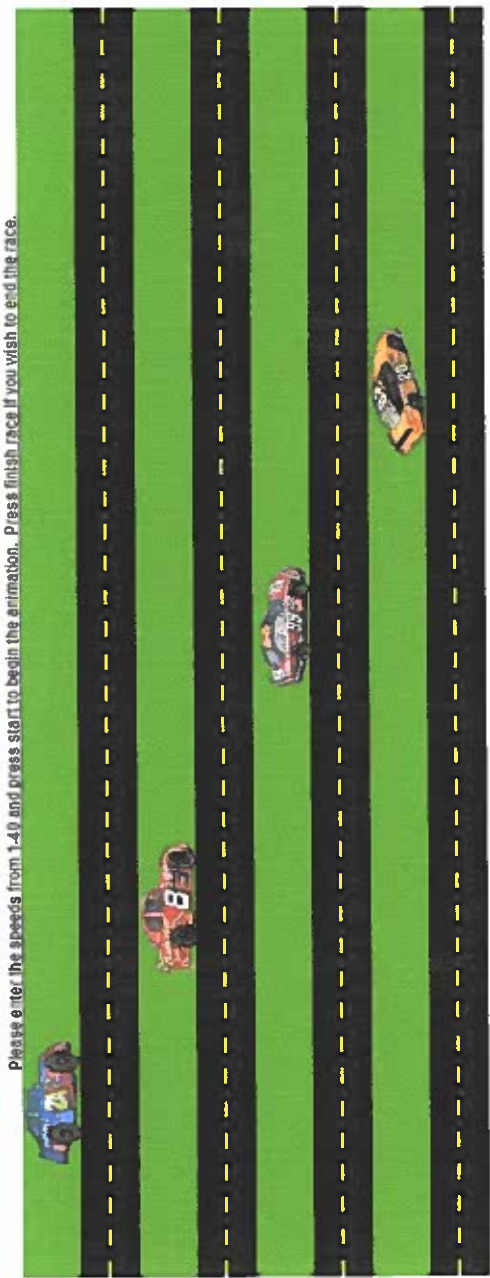
File Edit View History Bookmarks Tools Help

file:///E:/Seagate Sync/VOL/Personal folder/Documents/Senior Portfolio/Coding Projects/Coins 325 Java/erc/Daytona.htm

Most Visited Getting Started Latest Headlines

Welcome to the daytona 500!

Speed for the 24 car
 Speed for the 8 car
 Speed for the 99 car
 Speed for the 20 car



// Eric Foster

// Daytona

// 4/18/2008

// 4/18/2008

// Java I

// Grade: 100

// I used pictures, and buffered them across the background, moving them as

// a speed based off of the fps.

import java.applet.AudioClip;

import java.applet.Applet;

import java.lang.Math;

import java.awt.event.ActionListener;

import java.awt.event.ItemListener;

import java.awt.event.ItemEvent;

import java.awt.event.ActionEvent;

import java.awt.FlowLayout;

import java.awt.Color;

import java.awt.Graphics;

import java.util.Random;

import javax.swing.JFrame;

import javax.swing.JTextField;

import javax.swing.JLabel;

import javax.swing.JButton;

import javax.swing.JApplet;

import javax.swing.ImageIcon;

import java.awt.Image;

```
import java.awt.Container;
import javax.swing.Timer;
import java.awt.*;
import java.applet.Applet;

public class Daytona extends Applet implements Runnable
{
    int frameNumber;

    int delay;

    boolean startaction=false;

    Thread animatorThread;

    private Image jeff; // Jeff Gordon Car
    private Image dale; // Dale Earnhardt Car
    private Image edwards; // The Tide Car
    private Image tony; // Tony Stewart Car
    private Image track; // Track under cars
    private Image green; // Grass Behind Cars


    private JButton startJButton; // start button
    private JButton endJButton; // end race button and declares winner!


    private JTextField car1InputJTextField; // speed input for car 1
    private JTextField car2InputJTextField; // speed input for car 2
    private JTextField car3InputJTextField; // speed input for car 3
    private JTextField car4InputJTextField; // speed input for car 4


    private JLabel message1JLabel; // marks the speed for car 1
    private JLabel message2JLabel; // marks the speed for car 2
```



```
private JLabel message3JLabel; // marks the speed for car 3
private JLabel message4JLabel; // marks the speed for car 4
private JLabel message5JLabel; // informs the user that they must hit start to begin the
animation
```

```
public int car1speed=0; // speed of car 1;
public int car2speed=0; // speed of car 2;
public int car3speed=0; // speed of car 3;
public int car4speed=0; // speed of car 4;
```

```
private AudioClip sound1, sound2;
```

```
public void init()
```

```
{
```

```
String str = getParameter("fps");
int fps = (str != null) ? Integer.parseInt(str) : 10;
delay = (fps > 0) ? (100/ fps) : 100;
```

```
setLayout( new FlowLayout() );
```

```
ButtonHandler handler = new ButtonHandler();
```

```
jeff = getImage( getDocumentBase(), "jeffgordon.jpg" ); // 100 x 42 pixels
dale = getImage( getDocumentBase(), "dalejr.jpg" ); // 100 x 42 pixels
edwards = getImage( getDocumentBase(), "edwards.jpg" ); // 100 x 42 pixels
tony = getImage( getDocumentBase(), "tonystewart.jpg" );// 100 x 42 pixels
track = getImage( getDocumentBase(), "road.jpg" ); // 1000 x 42 pixels
green = getImage( getDocumentBase(), "grass.jpg" ); // 1000 x 42 pixel
```

```
sound1 = getAudioClip( getDocumentBase(), "cartcar.wav" );  
sound2 = getAudioClip( getDocumentBase(), "applause.wav" );
```

```
startJButton = new JButton( "Start");  
add( startJButton );  
startJButton.addActionListener( handler );
```

```
endJButton = new JButton( "Finish Race");  
add( endJButton );  
endJButton.addActionListener( handler);
```

```
message1JLabel = new JLabel( "Speed for the 24 car");  
add( message1JLabel );
```

```
car1InputJTextField = new JTextField( 5 ); // to enter speed  
add( car1InputJTextField );
```

```
message2JLabel = new JLabel( "Speed for the 8 car");  
add( message2JLabel );
```

```
car2InputJTextField = new JTextField( 5 ); // to enter speed  
add( car2InputJTextField );
```

```
message3JLabel = new JLabel( "Speed for the 99 car");  
add( message3JLabel );
```

```
car3InputJTextField = new JTextField( 5 ); // to enter speed
```

```
add( car3InputJTextField );
```

```
message4JLabel = new JLabel( "Speed for the 20 car" );
```

```
add( message4JLabel );
```

```
car4InputJTextField = new JTextField( 5 ); // to enter speed
```

```
add( car4InputJTextField );
```

```
message5JLabel = new JLabel( "Please enter the speeds from 1-40 and press start  
to begin the animation. Press finish race if you wish to end the race.");
```

```
add ( message5JLabel);
```

```
}
```

```
public void start()
```

```
{
```

```
    if (animatorThread == null)
```

```
    {
```

```
        animatorThread = new Thread(this);
```

```
        animatorThread.start();
```

```
    }
```

```
}
```

```
public void stop()
```

```
{
```

```
    animatorThread = null;
```

```
}
```

```

public void run()
{
    long startTime = System.currentTimeMillis();
    while (Thread.currentThread() == animatorThread)
    {
        repaint();
        try
        {
            startTime += delay;
            Thread.sleep(Math.max(0,
                                   startTime-System.currentTimeMillis()));
        } catch (InterruptedException e) {
            break;
        }

        frameNumber++;
    }
}

public void react( int speed, int speedy, int speedo, int speedi )
{
    car1speed=speed;
    car2speed=speedy;
    car3speed=speedo;
    car4speed=speedi;
}

```

```
private class ButtonHandler implements ActionListener
```

```
{
```

```
    public void actionPerformed( ActionEvent actionEvent)
```

```
    {
```

```
        if ( actionEvent.getSource() == startJButton )
```

```
        {
```

```
            startaction=true;
```

```
            sound1.loop();
```

```
            int speed1 = Integer.parseInt(car1InputJTextField.getText() );
```

```
            int speed2 = Integer.parseInt(car2InputJTextField.getText() );
```

```
            int speed3 = Integer.parseInt(car3InputJTextField.getText() );
```

```
            int speed4 = Integer.parseInt(car4InputJTextField.getText() );
```

```
            react( speed1, speed2, speed3, speed4 );
```

```
        }
```

```
        else if ( actionEvent.getSource() == endJButton )
```

```
        {
```

```
            startaction=false;
```

```
            sound1.stop();
```

```
            sound2.play();
```

```
        }
```

```
    }
```

```
}
```

```
public void update(Graphics g)
```

```
{
```

```
Dimension d = size();
```

```
int w1 = jeff.getWidth(this);
```

```
int h1 = jeff.getHeight(this);
```

```
int w2 = dale.getWidth(this);
```

```
int h2 = dale.getHeight(this);
```

```
int w3 = edwards.getWidth(this);
```

```
int h3 = edwards.getHeight(this);
```

```
int w4 = tony.getWidth(this);
```

```
int h4 = tony.getHeight(this);
```

```
if ((w1>0) && (h1>0) && (w2>0) && (h2>0) && (w3>0) && (h3>0) && (w4>0) &&  
(h4>0) && (startaction==true))
```

```
{
```

```
    g.drawImage(track, 0, 92, this);
```

```
    g.drawImage(green, 0, 50, this);
```

```
    g.drawImage(jeff,((frameNumber*car1speed) % (w1 + d.width)) - w1, 50, this);
```

```
    g.drawImage( green, 0, 134, this );
```

```
    g.drawImage( track, 0, 176, this );
```

```
    g.drawImage( dale, ((frameNumber*car2speed) % (w2 + d.width)) - w2, 134, this
```

```
);
```

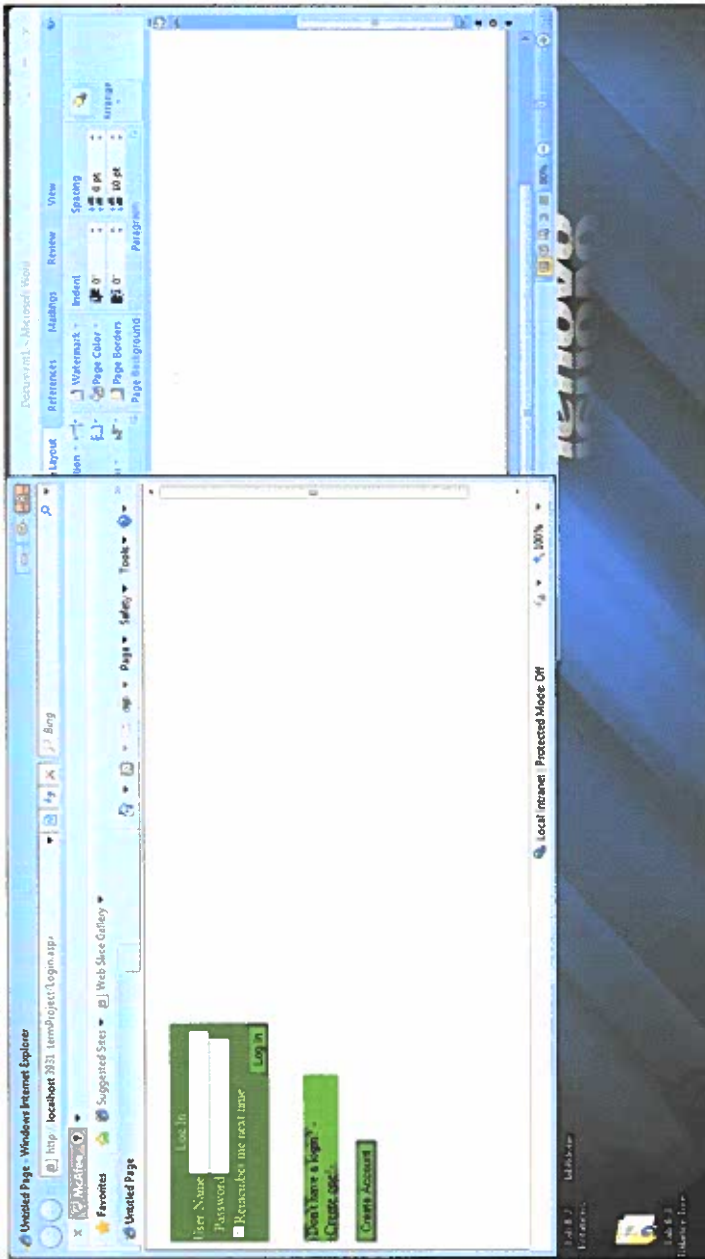
```
g.drawImage( green, 0, 218, this );  
g.drawImage( track, 0, 260, this );  
g.drawImage( edwards, ((frameNumber*car3speed) % (w3 + d.width)) - w3, 218,  
this );
```

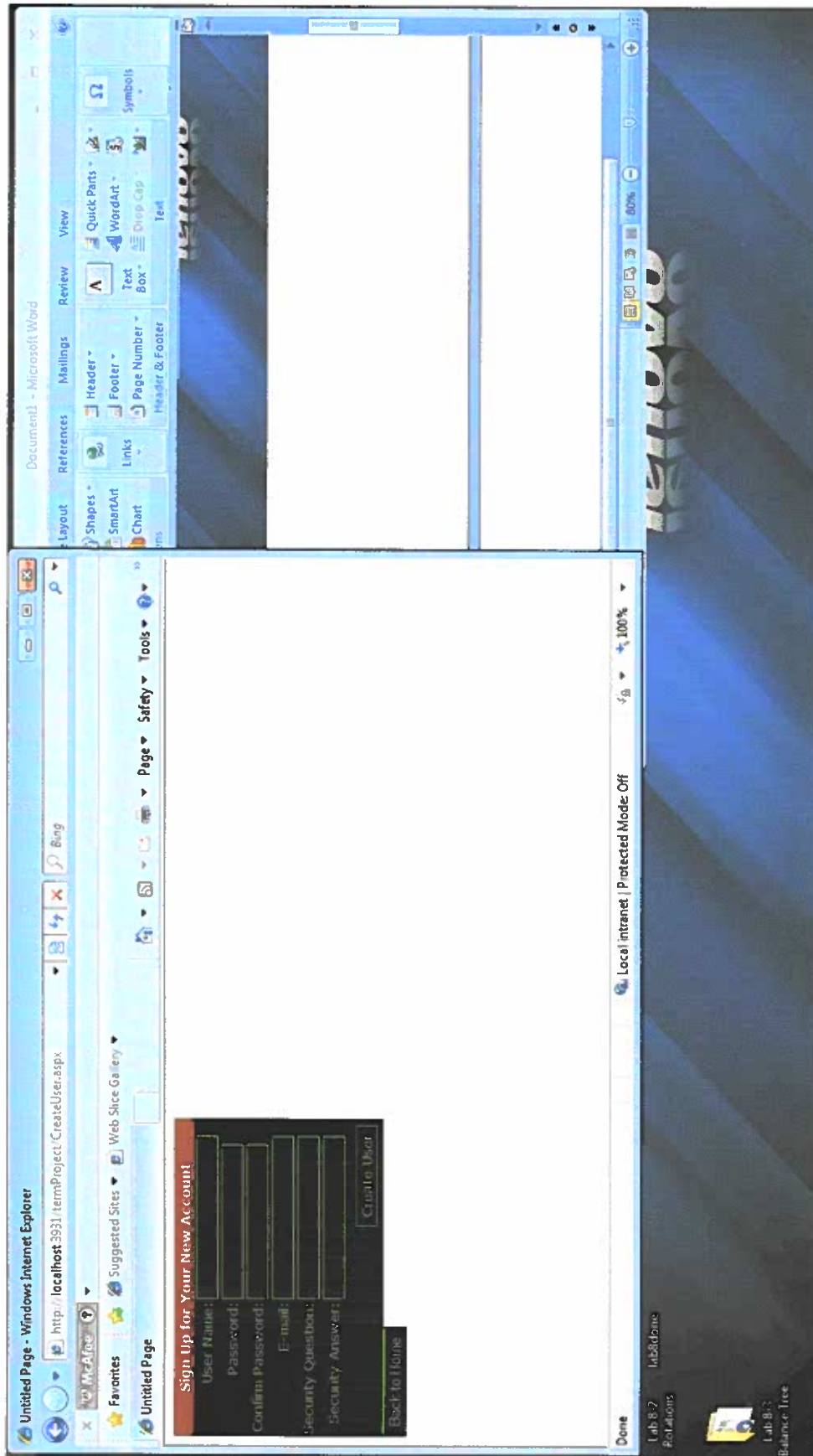
```
g.drawImage( green, 0, 302, this );  
g.drawImage( track, 0, 344, this );  
g.drawImage( tony, ((frameNumber*car4speed) % (w4 + d.width)) - w4, 302, this  
);
```

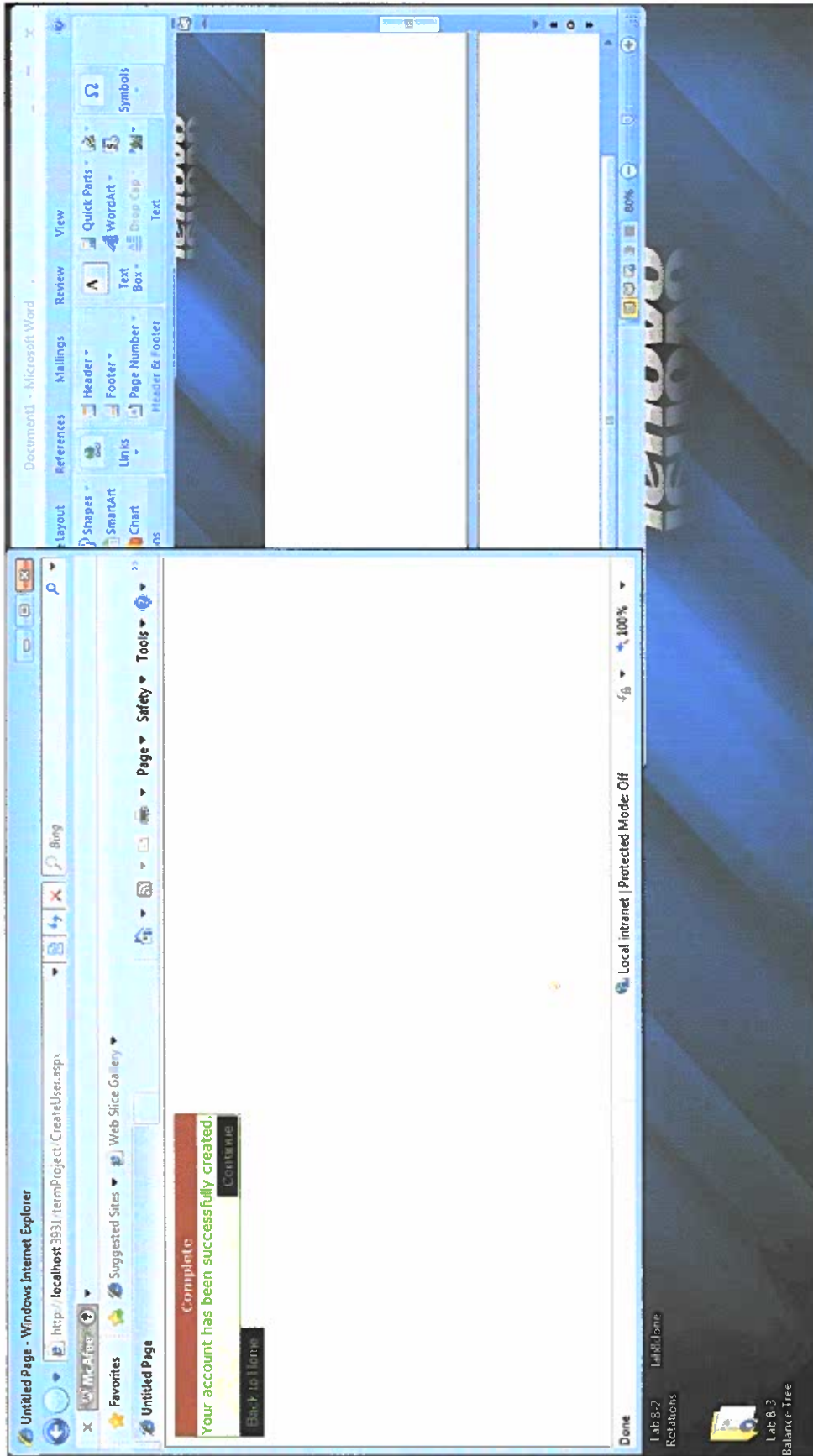
```
}
```

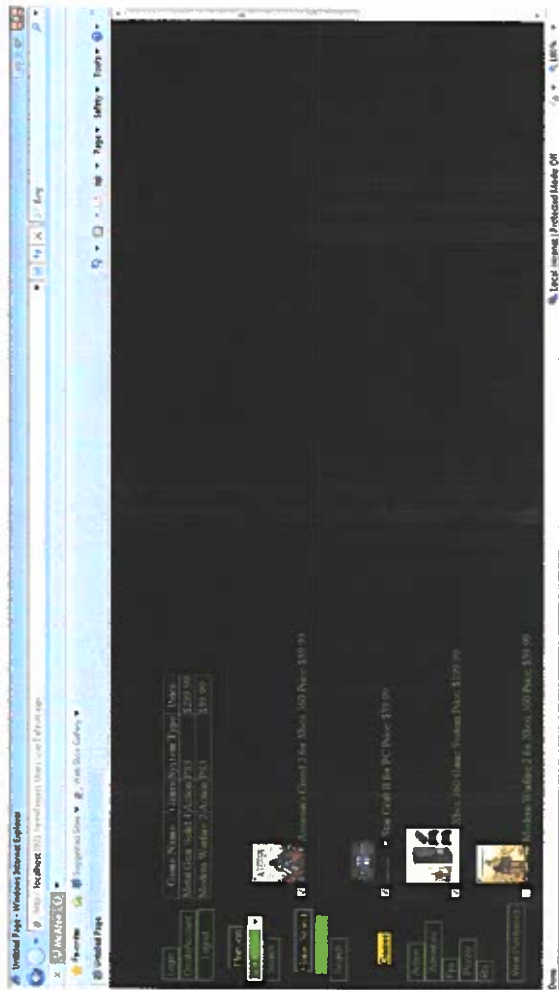
```
}
```

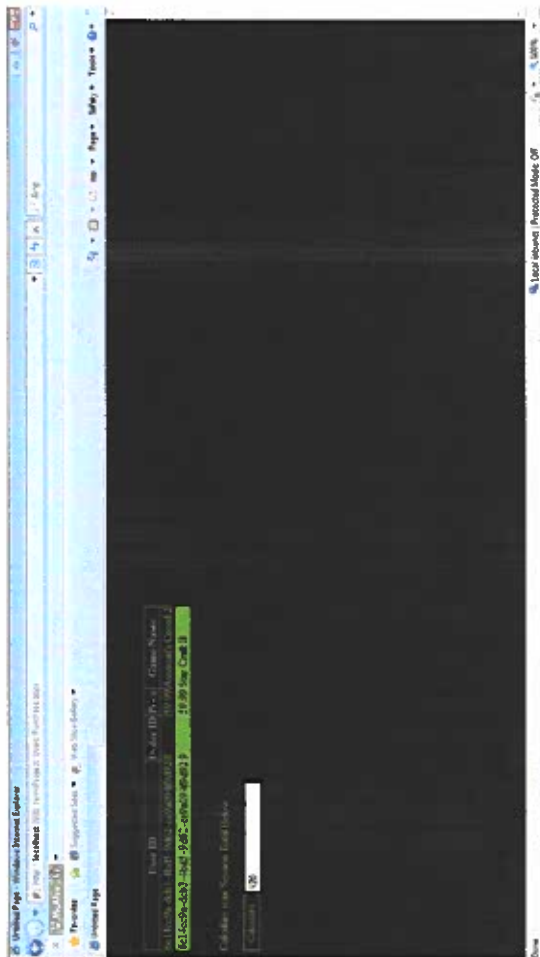
```
}
```











```

<%@ Page Language="VB" AutoEventWireup="false" CodeFile="CreateUser.aspx.vb"
Inherits="CreateUser" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml" >
<head runat="server">
    <title>Untitled Page</title>
</head>
<body>
    <form id="form1" runat="server">
        <div>
            <asp:CreateUserWizard ID="CreateUserWizard1" runat="server"
ContinueDestinationPageUrl="Login.aspx" BackColor="Black" BorderColor="Lime"
BorderStyle="Solid" BorderWidth="1px" Font-Names="Verdana" Font-Size="0.8em"
ForeColor="Lime">
                <WizardSteps>
                    <asp:CreateUserWizardStep runat="server">
                        <ContentTemplate>
                            <table border="0" style="font-size: 100%; color:
lime; font-family: Verdana; background-color: black">
                                <tr>
                                    <td align="center" colspan="2" style="font-
weight: bold; color: white; background-color: #990000">
                                        Sign Up for Your New Account</td>
                                </tr>
                                <tr>
                                    <td align="right">
                                        <asp:Label ID="UserNameLabel"
runat="server" AssociatedControlID="UserName">User Name:</asp:Label></td>
                                    <td>
                                        <asp:TextBox ID="UserName" runat="server"
BackColor="Black" BorderColor="Lime" ForeColor="Lime"></asp:TextBox>
                                        <asp:RequiredFieldValidator
ID="UserNameRequired" runat="server" BackColor="Black"
                                        BorderColor="Lime"
                                        ControlToValidate="UserName" ErrorMessage="User Name is required."
                                        ForeColor="Lime" ToolTip="User Name
is required."
                                        ValidationGroup="CreateUserWizard1">*</asp:RequiredFieldValidator>
                                    </td>
                                </tr>
                                <tr>
                                    <td align="right">
                                        <asp:Label ID="PasswordLabel"
runat="server" AssociatedControlID="Password">Password:</asp:Label></td>
                                    <td>
                                        <asp:TextBox ID="Password" runat="server"
BackColor="Black" BorderColor="Lime" ForeColor="Lime"
                                        TextMode="Password"></asp:TextBox>
                                        <asp:RequiredFieldValidator
ID="PasswordRequired" runat="server" BackColor="Black"
                                        BorderColor="Lime"
                                        ControlToValidate="Password" ErrorMessage="Password is required."
                                        ForeColor="Lime" ToolTip="Password is
required." ValidationGroup="CreateUserWizard1">*</asp:RequiredFieldValidator>
                                    </td>
                                </tr>
                            </table>
                        </ContentTemplate>
                    </asp:CreateUserWizardStep>
                </WizardSteps>
            </asp:CreateUserWizard>
        </div>
    </form>
</body>
</html>

```

```

        </td>
    </tr>
    <tr>
        <td align="right">
            <asp:Label ID="ConfirmPasswordLabel"
runat="server" AssociatedControlID="ConfirmPassword">Confirm
Password:</asp:Label></td>
            <td>
                <asp:TextBox ID="ConfirmPassword"
runat="server" BackColor="Black" BorderColor="Lime"
                    ForeColor="Lime"
TextMode="Password"></asp:TextBox>
                <asp:RequiredFieldValidator
ID="ConfirmPasswordRequired" runat="server" BackColor="Black"
                    BorderColor="Lime"
ControlToValidate="ConfirmPassword" ErrorMessage="Confirm Password is
required."
                    ForeColor="Lime" ToolTip="Confirm
Password is required."
ValidationGroup="CreateUserWizard1">*</asp:RequiredFieldValidator>
            </td>
    </tr>
    <tr>
        <td align="right">
            <asp:Label ID="EmailLabel" runat="server"
AssociatedControlID="Email">E-mail:</asp:Label></td>
            <td>
                <asp:TextBox ID="Email" runat="server"
BackColor="Black" BorderColor="Lime" ForeColor="Lime"></asp:TextBox>
                <asp:RequiredFieldValidator
ID="EmailRequired" runat="server" BackColor="Black" BorderColor="Lime"
                    ControlToValidate="Email"
ErrorMessage="E-mail is required." ForeColor="Lime"
                    ToolTip="E-mail is required."
ValidationGroup="CreateUserWizard1">*</asp:RequiredFieldValidator>
            </td>
    </tr>
    <tr>
        <td align="right">
            <asp:Label ID="QuestionLabel"
runat="server" AssociatedControlID="Question">Security
Question:</asp:Label></td>
            <td>
                <asp:TextBox ID="Question" runat="server"
BackColor="Black" BorderColor="Lime" ForeColor="Lime"></asp:TextBox>
                <asp:RequiredFieldValidator
ID="QuestionRequired" runat="server" BackColor="Black"
                    BorderColor="Lime"
ControlToValidate="Question" ErrorMessage="Security question is required."
                    ForeColor="Lime" ToolTip="Security
question is required."
ValidationGroup="CreateUserWizard1">*</asp:RequiredFieldValidator>
            </td>
    </tr>
    <tr>
        <td align="right">

```

```

                                <asp:Label ID="AnswerLabel"
runat="server" AssociatedControlID="Answer">Security Answer:</asp:Label></td>
                                <td>
                                <asp:TextBox ID="Answer" runat="server"
BackColor="Black" BorderColor="Lime" ForeColor="Lime"></asp:TextBox>
                                <asp:RequiredFieldValidator
ID="AnswerRequired" runat="server" BackColor="Black"
                                BorderColor="Lime"
ControlToValidate="Answer" ErrorMessage="Security answer is required."
                                ForeColor="Lime" ToolTip="Security
answer is required."
ValidationGroup="CreateUserWizard1">*</asp:RequiredFieldValidator>
                                </td>
                                </tr>
                                <tr>
                                <td align="center" colspan="2">
                                <asp:CompareValidator
ID="PasswordCompare" runat="server" BackColor="Black" BorderColor="Lime"
                                ControlToCompare="Password"
ControlToValidate="ConfirmPassword" Display="Dynamic"
                                ErrorMessage="The Password and
Confirmation Password must match." ForeColor="Lime"
ValidationGroup="CreateUserWizard1"></asp:CompareValidator>
                                </td>
                                </tr>
                                <tr>
                                <td align="center" colspan="2" style="color:
red">
                                <asp:Literal ID="ErrorMessage"
runat="server" EnableViewState="False"></asp:Literal>
                                </td>
                                </tr>
                                </table>
                                </ContentTemplate>
                                </asp:CreateUserWizardStep>
                                <asp:CompleteWizardStep runat="server">
                                <ContentTemplate>
                                <table border="0" style="font-size: 100%; font-
family: Verdana; background-color: #ffffd6">
                                <tr>
                                <td align="center" colspan="2" style="font-
weight: bold; color: white; height: 18px;
                                background-color: #990000">
                                Complete</td>
                                </tr>
                                <tr>
                                <td>
                                Your account has been successfully
created.</td>
                                </tr>
                                <tr>
                                <td align="right" colspan="2">
                                <asp:Button ID="ContinueButton"
runat="server" BackColor="Black" BorderColor="Lime"
                                BorderStyle="Solid" BorderWidth="1px"
CausesValidation="False" CommandName="Continue"

```



```
<%@ Page Language="VB" AutoEventWireup="false" CodeFile="Default.aspx.vb"
Inherits="_Default" %>
```

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
```

```
<html xmlns="http://www.w3.org/1999/xhtml" >
<head runat="server">
    <title>Untitled Page</title>
</head>
<body bgcolor="black">
    <form id="form1" runat="server">
        <div>
            <div style="width: 112px; height: 464px; float:left;">
                <asp:Button ID="Button2" runat="server" BackColor="Black"
BorderColor="Lime" ForeColor="Lime"
                    Text="Login" />&nbsp;
                <asp:Button ID="Button1" runat="server" BackColor="Black"
BorderColor="Lime" Text="CreateAccount"
                    Width="101px" ForeColor="Lime" /><br />
                <br />
                <br />
                &nbsp;&nbsp;&nbsp;&nbsp;&nbsp;<asp:Label ID="Label1" runat="server"
BackColor="Black" BorderColor="Green"
                    BorderStyle="Groove" Text="Platform:"
ForeColor="Yellow"></asp:Label>
                <asp:DropDownList ID="DropDownList1" runat="server"
BackColor="Lime" ForeColor="Black">
                    <asp:ListItem>Xbox 360</asp:ListItem>
                    <asp:ListItem>PS3</asp:ListItem>
                    <asp:ListItem>Wii</asp:ListItem>
                    <asp:ListItem>PC</asp:ListItem>
                    <asp:ListItem>Xbox</asp:ListItem>
                    <asp:ListItem>PS2</asp:ListItem>
                    <asp:ListItem>GameCube</asp:ListItem>
                </asp:DropDownList>&nbsp;<br />
                <asp:Button ID="Button3" runat="server" BackColor="Black"
BorderColor="Lime" ForeColor="Lime"
                    Text="Search" /><br />
                <br />
                &nbsp;&nbsp;&nbsp;&nbsp;&nbsp;<asp:Label ID="Label2" runat="server" BackColor="Black"
BorderColor="Teal" BorderStyle="Groove"
                    Text="Game Search" ForeColor="Yellow"></asp:Label><br />
                &nbsp;&nbsp;&nbsp;&nbsp;&nbsp;<asp:TextBox ID="TextBox1" runat="server" BackColor="Lime"
BorderColor="Black" BorderStyle="Outset"
                    ForeColor="Black" Width="87px"></asp:TextBox><br />
                <asp:Button ID="Button4" runat="server" BackColor="Black"
BorderColor="Lime" ForeColor="Lime"
                    Text="Search" /><br />
                <br />
                <br />
                &nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~
            </div>
        </div>
    </form>
</body>
</html>
```

```

                                Font-Names="Verdana" ForeColor="Lime"
Text="Continue" ValidationGroup="CreateUserWizard1" />
                                </td>
                                </tr>
                                </table>
                                </ContentTemplate>
                                </asp:CompleteWizardStep>
                                </WizardSteps>
                                <FinishCompleteButtonStyle BackColor="Black" BorderColor="Lime"
/>
                                <SideBarStyle BackColor="#990000" Font-Size="0.9em"
VerticalAlign="Top" />
                                <InstructionTextStyle BackColor="Black" BorderColor="Lime"
ForeColor="Lime" />
                                <SideBarButtonStyle ForeColor="White" />
                                <ContinueButtonStyle BackColor="Black" BorderColor="Lime"
BorderStyle="Solid" BorderWidth="1px"
                                Font-Names="Verdana" ForeColor="#990000" />
                                <ValidatorTextStyle BackColor="Black" BorderColor="Lime"
ForeColor="Lime" />
                                <NavigationButtonStyle BackColor="White" BorderColor="#CC9966"
BorderStyle="Solid"
                                BorderWidth="1px" Font-Names="Verdana" ForeColor="#990000" />
                                <HeaderStyle BackColor="Black" BorderColor="Lime"
BorderStyle="Solid" BorderWidth="2px"
                                Font-Bold="True" Font-Size="0.9em" ForeColor="Lime"
HorizontalAlign="Center" />
                                <TextBoxStyle BackColor="Black" BorderColor="Lime"
ForeColor="Lime" />
                                <LabelStyle BackColor="Black" BorderColor="Lime" ForeColor="Lime"
/>
                                <CreateUserButtonStyle BackColor="Black" BorderColor="Lime"
BorderStyle="Solid" BorderWidth="1px"
                                Font-Names="Verdana" ForeColor="Lime" />
                                <ErrorMessageStyle BackColor="Black" BorderColor="Lime"
ForeColor="Lime" />
                                <TitleTextStyle BackColor="Black" BorderColor="Lime" Font-
Bold="True" ForeColor="Lime" />
                                <HyperLinkStyle BackColor="Black" BorderColor="Lime"
ForeColor="Lime" />
                                <FinishPreviousButtonStyle BackColor="Black" BorderColor="Lime"
/>
                                <StepStyle BackColor="Black" BorderColor="Lime" ForeColor="Lime"
/>
                                <CancelButtonStyle BackColor="Black" BorderColor="Lime"
ForeColor="Lime" />
                                </asp:CreateUserWizard>
                                <asp:Button ID="Button1" runat="server" Text="Back to Home"
BackColor="Black" BorderColor="Lime" ForeColor="Lime" /></div>
        </form>
</body>
</html>

```

[illegible]


```

    </div>
    <br />
    &nbsp;<br />
    <asp:Label ID="Label1" runat="server" BackColor="Lime"
BorderColor="Lime" ForeColor="Black"
    Text="-Don't have a login? -</br>-Create one!-"
Width="182px"></asp:Label><br />
    <br />
    <asp:Button ID="Button1" runat="server" BackColor="#00C000"
BorderColor="#004000"
    Text="Create Account" /><br />
    <br />
    <br />
    <br />
    <br />
    <br />
    <br />
    <br />
    <br />
    <br />
    <br />
    <br />
    <asp:Button ID="Button2" runat="server" BackColor="#00C000"
BorderColor="#004000"
    Text="Back to Home" />
</form>
</body>
</html>

```

```

<%@ Page Language="VB" AutoEventWireup="false" CodeFile="Purchase.aspx.vb"
Inherits="Purchase" %>

```

```

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

```

```

<html xmlns="http://www.w3.org/1999/xhtml" >

```

```

<head runat="server">

```

```

    <title>Untitled Page</title>

```

```

</head>

```

```

<body bgcolor="black">

```

```

    <form id="form1" runat="server">

```

```

        <div>

```

```

            <asp:GridView ID="GridView1" runat="server"
AutoGenerateColumns="False"

```

```

                BackColor="Black" BorderColor="Lime" DataKeyNames="Order ID"
DataSourceID="SqlDataSource1"
                ForeColor="Lime">

```

```

                    <Columns>

```

```

                        <asp:BoundField DataField="User ID" HeaderText="User ID"
SortExpression="User ID" />

```

```

                        <asp:BoundField DataField="Order ID" HeaderText="Order ID"
InsertVisible="False"

```